

TODAS LAS PRACTICAS Y SEMINARIOS...



user_2492681



Sistemas Concurrentes y Distribuidos



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada

EOI Escuela de
organización
industrial

La mejor escuela de negocios en
energía, sostenibilidad y medio
ambiente de España.

Más información
www.eoi.es

Formamos
talento para un futuro
Sostenible



100% Empleabilidad



Modalidad: Presencial u online



**Programa de Becas,
Bonificaciones y Descuentos**

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH



SISTEMAS CONCURRENTES Y DISTRIBUIDOS

Todas las actividades de seminarios y prácticas resueltas.

Índice:

- **Seminario 1. Programación multihebra y semáforos.**
 - 1. Cálculo concurrente de una integral 1
- **Práctica 1. Sincronización de hebras con semáforos.**
 - 1. Problema del productor-consumidor (buffer LIFO) 3
 - 2. Problema del productor-consumidor (buffer FIFO) 6
 - 3. Problema de los múltiples productores-consumidores (buffer LIFO) 9
 - 4. Problema de los múltiples productores-consumidores (buffer FIFO) 12
 - 5. Problema de los fumadores 15
- **Seminario 2. Introducción a los monitores en C++11.**
 - 1. monitor_em.cpp 17
 - 2. Problema del productor-consumidor SU (buffer FIFO) 20
- **Práctica 2. Casos prácticos de monitores en C++11.**
 - 1. Problema de los múltiples productores-consumidores (buffer LIFO, monitor SU) 25
 - 2. Problema de los múltiples productores-consumidores (buffer FIFO, monitor SU) 30
 - 3. Problema de los fumadores (monitor SU) 35
 - 4. Problema de los lectores y escritores (monitor SU) 39
- **Práctica 3. Implementación de algoritmos distribuidos con MPI.**
 - 1. Problema de los múltiples productores-consumidores (buffer FIFO) 42
 - 2. Cena de los filósofos (primera aproximación con interbloqueo) 46
 - 3. Cena de los filósofos con camarero 49
- **Práctica 4. Implementación de Sistemas de Tiempo Real.**
 - 1. Nueva funcionalidad en ejecutivo1.cpp para informar retrasos 53
 - 2. Nuevo ejemplo 55

WUOLAH

Seminario 1. Programación multihebra y semáforos.

1. Cálculo concurrente de una integral.

```
#include <iostream>
#include <iomanip>
#include <chrono> // incluye now, time\_point, duration
#include <future>
#include <vector>
#include <cmath>

using namespace std ;
using namespace std::chrono;

const long m = 1024l*1024l*1024l, // número de muestras (del orden de mil millones)
n = 4 ; // número de hebras concurrentes (divisor de 'm')

// -----
// evalua la función $f$ a integrar ($f(x)=4/(1+x^2)$)
double f(double x){
    return 4.0/(1.0+x*x) ;
}
// -----
// calcula la integral de forma secuencial, devuelve resultado:
double calcular_integral_secuencial( )
{
    double suma = 0.0 ; // inicializar suma
    for( long j = 0 ; j < m ; j++ ) // para cada $j$ entre $0$ y $m-1$:
    { const double xj = double(j+0.5)/m ; // calcular $x_j$
      suma += f( xj ) ; // añadir $f(x_j)$ a la suma actual
    }
    return suma/m ; // devolver valor promedio de $f$
}

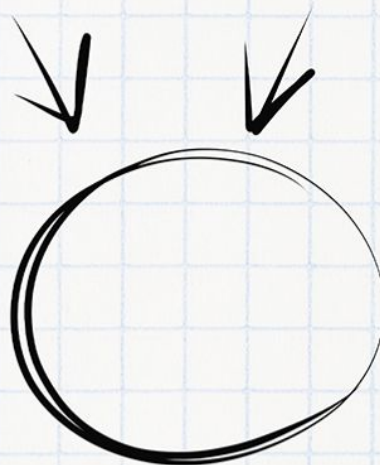
// -----
// función que ejecuta cada hebra: recibe $i$ ==índice de la hebra, ($0\leq i<n$)
double funcion_hebra( long i )
{
    double suma = 0.0;
    for(long k=i*m/n; k<i*m/n+m/n; k++){
        const double xj = double(k+0.5)/m ;
        suma += f( xj );
    }
    return suma;
}
```

Imagínate aprobando el examen

Necesitas tiempo y concentración

Planes	 PLAN TURBO	 PLAN PRO	 PLAN PRO+
 Descargas sin publi al mes	10 	40 	80 
 Elimina el video entre descargas			
 Descarga carpetas			
 Descarga archivos grandes			
 Visualiza apuntes online sin publi			
 Elimina toda la publi web			
 Precios Anual ☐	0,99 € / mes	3,99 € / mes	7,99 € / mes

Ahora que puedes conseguirlo,
¿Qué nota vas a sacar?



WUOLAH

Sistemas Concurrentes y Dist...



Comparte estos flyers en tu clase y consigue más dinero y recompensas



Banco de apuntes de la

WUOLAH

- 1** Imprime esta hoja
- 2** Recorta por la mitad
- 3** Coloca en un lugar visible para que tus compis puedan escanar y acceder a apuntes

- 4** Llévate dinero por cada descarga de los documentos descargados a través de tu QR




```
// -----
// calculo de la integral de forma concurrente
double calcular_integral_concurrente( )
{
    double suma = 0.0;
    future<double> futuros[n];
    for (long i=0; i<n; i++){
        futuros[i] = async(launch::async, funcion_hebra, i);
    }
    for (long i=0; i<n; i++){
        suma += futuros[i].get();
    }
    return suma/m;
}
// -----

int main()
{
    time_point<steady_clock> inicio_sec = steady_clock::now() ;
    const double      result_sec = calcular_integral_secuencial( );
    time_point<steady_clock> fin_sec = steady_clock::now() ;

    double x = sin(0.4567);
    time_point<steady_clock> inicio_conc = steady_clock::now() ;
    const double      result_conc = calcular_integral_concurrente( );
    time_point<steady_clock> fin_conc = steady_clock::now() ;
    duration<float,milli> tiempo_sec = fin_sec - inicio_sec ,
        tiempo_conc = fin_conc - inicio_conc ;
    const float      porc = 100.0*tiempo_conc.count()/tiempo_sec.count() ;

    constexpr double pi = 3.141592653589793238461 ;

    cout << "Número de muestras (m) : " << m << endl
        << "Número de hebras (n) : " << n << endl
        << setprecision(18)
        << "Valor de PI : " << pi << endl
        << "Resultado secuencial : " << result_sec << endl
        << "Resultado concurrente : " << result_conc << endl
        << setprecision(5)
        << "Tiempo secuencial : " << tiempo_sec.count() << " milisegundos. " << endl
        << "Tiempo concurrente : " << tiempo_conc.count() << " milisegundos. " << endl
        << setprecision(4)
        << "Porcentaje t.conc/t.sec. : " << porc << "%" << endl;
}

```

```
teresa@teresa-VirtualBox:~/Desktop/SCD-repaso/material/1a/scd-s1-fuentes$ g++ -std=c++11 -pthread -o integral integral.cpp scd.cpp
teresa@teresa-VirtualBox:~/Desktop/SCD-repaso/material/1a/scd-s1-fuentes$ ./integral
Número de muestras (m) : 1073741824
Número de hebras (n) : 4
Valor de PI : 3.14159265358979312
Resultado secuencial : 3.14159265358998185
Resultado concurrente : 3.14159265358982731
Tiempo secuencial : 14256 milisegundos.
Tiempo concurrente : 3057.8 milisegundos.
Porcentaje t.conc/t.sec. : 21.45%
```

2

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

Práctica 1. Sincronización de hebras con semáforos.

1. Problema del productor-consumidor (buffer LIFO)

```
#include <iostream>
#include <cassert>
#include <thread>
#include <mutex>
#include <random>
#include "scd.h"

using namespace std;
using namespace scd;

//*****
// Variables globales

const unsigned
    num_items = 40 , // número de items
    tam_vec = 10 ; // tamaño del buffer
unsigned
    cont_prod[num_items] = {0}, // contadores de verificación: para cada dato, número de
    veces que se ha producido.
    cont_cons[num_items] = {0}, // contadores de verificación: para cada dato, número de
    veces que se ha consumido.
    siguiente_dato = 0, // siguiente dato a producir en 'producir_dato' (solo se usa ahí)
    primera_libre = 0,
    buff[tam_vec];

Semaphore libres = tam_vec;
Semaphore ocupadas = 0;

//*****
// funciones comunes a las dos soluciones (fifo y lifo)
//-----

unsigned producir_dato(){
    this_thread::sleep_for( chrono::milliseconds( aleatorio<20,100>() ));
    const unsigned dato_producido = siguiente_dato;
    siguiente_dato++;
    cont_prod[dato_producido] ++;
    cout << "          producido: " << dato_producido << endl << flush ;
    return dato_producido ;
}

//-----
```

```

void consumir_dato( unsigned dato ){
    assert( dato < num_items );
    cont_cons[dato] ++;
    this_thread::sleep_for( chrono::milliseconds( aleatorio<20,100>() ) );
    cout << "                                consumido: " << dato << endl << flush;
}
//-----
void test_contadores(){
    bool ok = true;
    cout << "comprobando contadores ...." ;
    for( unsigned i = 0; i < num_items; i++ ){
        if (cont_prod[i] != 1){
            cout << "error: valor " << i << " producido " << cont_prod[i] << " veces." <<
endl;
            ok = false;
        }
        if (cont_cons[i] != 1){
            cout << "error: valor " << i << " consumido " << cont_cons[i] << " veces" <<
endl;
            ok = false;
        }
    }
    if (ok)
        cout << endl << flush << "solución (aparentemente) correcta." << endl << flush;
}
//-----
void funcion_hebra_productora(){
    for( unsigned i = 0; i < num_items; i++ ){
        int dato = producir_dato();
        sem_wait(libres);
        buff[primera_libre] = dato;
        cout << "+ insertado " << dato << endl << flush;
        primera_libre++;
        sem_signal(ocupadas);
    }
}
//-----
void funcion_hebra_consumidora(){
    for(unsigned i = 0; i < num_items; i++){
        int dato;
        sem_wait(ocupadas);
        primera_libre--;
        dato = buff[primera_libre];
        cout << "- extraído " << dato << endl << flush;
        sem_signal(libres);
        consumir_dato(dato);
    }
}

```



```
//-----
int main(){
    cout << "-----" << endl
    << "Problema de los productores-consumidores (solución LIFO)." << endl
    << "-----" << endl
    << flush;

    thread hebra_productora ( funcion_hebra_productora ), hebra_consumidora(
funcion_hebra_consumidora );

    hebra_productora.join();
    hebra_consumidora.join();

    cout << "fin" << endl;

    test_contadores();
}
```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

2. Problema del productor-consumidor (buffer FIFO)

```
#include <iostream>
#include <cassert>
#include <thread>
#include <mutex>
#include <random>
#include "scd.h"

using namespace std;
using namespace scd;
//*****
// Variables globales

const unsigned
    num_items = 40 , // número de items
    tam_vec  = 10 ; // tamaño del buffer
unsigned
    cont_prod[num_items] = {0}, // contadores de verificación: para cada dato, número de
    veces que se ha producido.
    cont_cons[num_items] = {0}, // contadores de verificación: para cada dato, número de
    veces que se ha consumido.
    siguiente_dato      = 0, // siguiente dato a producir en 'producir_dato' (solo se usa ahí)
    primera_libre       = 0,
    primera_ocupada     = 0,
    buff[tam_vec];

Semaphore libres = tam_vec;
Semaphore ocupadas = 0;

//*****
// funciones comunes a las dos soluciones (fifo y lifo)
//-----
unsigned producir_dato(){
    this_thread::sleep_for( chrono::milliseconds( aleatorio<20,120>() ));
    const unsigned dato_producido = siguiente_dato;
    siguiente_dato++;
    cont_prod[dato_producido] ++;
    cout << "          producido: " << dato_producido << endl << flush ;
    return dato_producido ;
}
//-----
void consumir_dato( unsigned dato ){
    assert( dato < num_items );
    cont_cons[dato] ++;
    this_thread::sleep_for( chrono::milliseconds( aleatorio<150,400>() ));
    cout << "          consumido: " << dato << endl << flush;
}
```

```
//-----

void test_contadores(){
    bool ok = true;
    cout << "comprobando contadores ...." ;
    for( unsigned i = 0; i < num_items; i++){
        if (cont_prod[i] != 1){
            cout << "error: valor " << i << " producido " << cont_prod[i] << " veces." <<
endl;
            ok = false;
        }
        if (cont_cons[i] != 1){
            cout << "error: valor " << i << " consumido " << cont_cons[i] << " veces" <<
endl;
            ok = false;
        }
    }
    if (ok)
        cout << endl << flush << "solución (aparentemente) correcta." << endl << flush;
}

//-----

void funcion_hebra_productora(){
    for( unsigned i = 0; i < num_items; i++){
        int dato = producir_dato();
        sem_wait(libres);
        buff[primera_libre] = dato;
        cout << "+ insertado " << dato << endl << flush;
        primera_libre = (primera_libre + 1) % tam_vec;
        sem_signal(ocupadas);
    }
}

//-----

void funcion_hebra_consumidora(){
    for( unsigned i = 0; i < num_items; i++){
        int dato;
        sem_wait(ocupadas);
        dato = buff[primera_ocupada];
        primera_ocupada = (primera_ocupada + 1) % tam_vec;
        cout << "- extraído " << dato << endl << flush;
        sem_signal(libres);
        consumir_dato(dato);
    }
}

//-----
```

```

int main(){
    cout << "-----" << endl
    << "Problema de los productores-consumidores (solución FIFO)." << endl
    << "-----" << endl
    << flush;

    thread hebra_productora ( funcion_hebra_productora ), hebra_consumidora(
funcion_hebra_consumidora );

    hebra_productora.join();
    hebra_consumidora.join();

    cout << "fin" << endl;

    test_contadores();
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

3. Problema de los múltiples productores y consumidores (buffer LIFO)

```
#include <iostream>
#include <cassert>
#include <thread>
#include <mutex>
#include <random>
#include "scd.h"

using namespace std;
using namespace scd;

//*****
// Variables globales

const unsigned
    m      = 40, // número de items
    tam_vec = 10, // tamaño del buffer
    np      = 5,  // número de productores
    nc      = 4;  // número de consumidores

unsigned
    cont_prod[m] = {0}, // contadores de verificación: para cada dato, número de veces
                        // que se ha producido.
    cont_cons[m] = {0}, // contadores de verificación: para cada dato, número de veces
                        // que se ha consumido.
    siguiente_dato = 0, // siguiente dato a producir en 'producir_dato' (solo se usa ahí)
    primera_libre = 0,
    p              = m/np, // número de items que produce cada productor
    c              = m/nc, // número de items que consume cada consumidor
    items_producidos[np] = {0},
    buff[tam_vec];

Semaphore libres = tam_vec;
Semaphore ocupadas = 0;

//*****
// funciones comunes a las dos soluciones (fifo y lifo)
//-----
unsigned producir_dato(int ip){
    assert(ip < np);
    this_thread::sleep_for( chrono::milliseconds( aleatorio<20,100>() ));
    const unsigned dato_producido = items_producidos[ip] + ip*p;
    items_producidos[ip]++;
    cont_prod[dato_producido]++;
    cout << "                               producido: " << dato_producido << " por la hebra " <<
    ip << endl << flush ;
    return dato_producido;
}
```



```
//-----

void consumir_dato(int ic, unsigned dato ){
    assert(dato < m);
    assert(ic < nc);
    cont_cons[dato]++;
    this_thread::sleep_for( chrono::milliseconds( aleatorio<20,100>() ));
    cout << "                                consumido: " << dato << " por la
hebra " << ic << endl << flush;
}
//-----

void test_contadores(){
    bool ok = true;
    cout << "comprobando contadores ...." ;
    for( unsigned i = 0; i < m; i++ ){
        if (cont_prod[i] != 1){
            cout << "error: valor " << i << " producido " << cont_prod[i] << " veces." <<
endl;
            ok = false;
        }
        if (cont_cons[i] != 1){
            cout << "error: valor " << i << " consumido " << cont_cons[i] << " veces" <<
endl;
            ok = false;
        }
    }
    if (ok)
        cout << endl << flush << "solución (aparentemente) correcta." << endl << flush;
}

//-----

void funcion_hebra_productora(int ih){
    for(unsigned i = 0; i < p; i++ ){
        int dato = producir_dato(ih);
        sem_wait(libres);
        buff[primera_libre] = dato;
        cout << "+ insertado " << dato << " por hebra " << ih << endl << flush;
        primera_libre++;
        sem_signal(ocupadas);
    }
}

//-----
```

```

void funcion_hebra_consumidora(int ih){
    for(unsigned i = 0; i < c; i++){
        int dato;
        sem_wait(ocupadas);
        primera_libre--;
        dato = buff[primera_libre];
        cout << "- extraído " << dato << " por hebra " << ih << endl << flush;
        sem_signal(libres);
        consumir_dato(ih, dato);
    }
}

//-----

int main(){
    cout << "-----" << endl
    << "Problema de los productores-consumidores múltiples (solución LIFO)." << endl
    << "-----" << endl
    << flush;
    cout << "Número de productores: " << np << endl
    << "Número de consumidores: " << nc << endl
    << "Número de items: " << m << endl << flush;

    thread hebras_productoras[np], hebras_consumidoras[nc];

    for (int i=0; i<np; i++){
        hebras_productoras[i] = thread(funcion_hebra_productora, i);
    }
    for (int i=0; i<nc; i++){
        hebras_consumidoras[i] = thread(funcion_hebra_consumidora, i);
    }

    for (int i=0; i<np; i++){
        hebras_productoras[i].join();
    }
    for (int i=0; i<nc; i++){
        hebras_consumidoras[i].join();
    }

    cout << "fin" << endl;
    test_contadores();
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



4. Problema de los múltiples productores y consumidores (buffer FIFO)

```
#include <iostream>
#include <cassert>
#include <thread>
#include <mutex>
#include <random>
#include "scd.h"

using namespace std;
using namespace scd;
//*****
// Variables globales

const unsigned
    m      = 40, // número de items
    tam_vec = 10, // tamaño del buffer
    np      = 5,  // número de productores
    nc      = 4;  // número de consumidores

unsigned
    cont_prod[m] = {0}, // contadores de verificación: para cada dato, número de veces
                        // que se ha producido.
    cont_cons[m] = {0}, // contadores de verificación: para cada dato, número de veces
                        // que se ha consumido.
    siguiente_dato = 0, // siguiente dato a producir en 'producir_dato' (solo se usa ahí)
    primera_libre  = 0,
    primera_ocupada = 0,
    p              = m/np, // número de items que produce cada productor
    c              = m/nc, // número de items que consume cada consumidor
    items_producidos[np] = {0},
    buff[tam_vec];

Semaphore libres = tam_vec;
Semaphore ocupadas = 0;

//*****
// funciones comunes a las dos soluciones (fifo y lifo)
//-----
unsigned producir_dato(int ip){
    assert(ip < np);
    this_thread::sleep_for( chrono::milliseconds( aleatorio<20,100>() ));
    const unsigned dato_producido = items_producidos[ip] + ip*p;
    items_producidos[ip]++;
    cont_prod[dato_producido]++;
    cout << "                                producido: " << dato_producido << " por la hebra " <<
    ip << endl << flush ;
    return dato_producido;
}
```

Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
//-----

void consumir_dato(int ic, unsigned dato ){
    assert(dato < m);
    assert(ic < nc);
    cont_cons[dato]++;
    this_thread::sleep_for( chrono::milliseconds( aleatorio<20,100>() ));
    cout << "                                consumido: " << dato << " por la
hebra " << ic << endl << flush;
}

//-----

void test_contadores(){
    bool ok = true;
    cout << "comprobando contadores ...." ;
    for( unsigned i = 0; i < m; i++ ){
        if (cont_prod[i] != 1){
            cout << "error: valor " << i << " producido " << cont_prod[i] << " veces." <<
endl;
            ok = false;
        }
        if (cont_cons[i] != 1){
            cout << "error: valor " << i << " consumido " << cont_cons[i] << " veces" <<
endl;
            ok = false;
        }
    }
    if (ok)
        cout << endl << flush << "solución (aparentemente) correcta." << endl << flush;
}

//-----

void funcion_hebra_productora(int ih){
    for(unsigned i = 0; i < p; i++ ){
        int dato = producir_dato(ih);
        sem_wait(libres);
        buff[primera_libre] = dato;
        cout << "+ insertado " << dato << " por hebra " << ih << endl << flush;
        primera_libre = (primera_libre + 1) % tam_vec;
        sem_signal(ocupadas);
    }
}

//-----
```

```

void funcion_hebra_consumidora(int ih){
    for(unsigned i = 0; i < c; i++){
        int dato;
        sem_wait(ocupadas);
        dato = buff[primera_ocupada];
        primera_ocupada = (primera_ocupada + 1) % tam_vec;
        cout << "- extraído " << dato << " por hebra " << ih << endl << flush;
        sem_signal(libres);
        consumir_dato(ih, dato);
    }
}

//-----

int main(){
    cout << "-----" << endl
    << "Problema de los productores-consumidores múltiples (solución FIFO)." << endl
    << "-----" << endl
    << flush;
    cout << "Número de productores: " << np << endl
    << "Número de consumidores: " << nc << endl
    << "Número de items: " << m << endl << flush;

    thread hebras_productoras[np], hebras_consumidoras[nc];

    for (int i=0; i<np; i++){
        hebras_productoras[i] = thread(funcion_hebra_productora, i);
    }
    for (int i=0; i<nc; i++){
        hebras_consumidoras[i] = thread(funcion_hebra_consumidora, i);
    }

    for (int i=0; i<np; i++){
        hebras_productoras[i].join();
    }
    for (int i=0; i<nc; i++){
        hebras_consumidoras[i].join();
    }

    cout << "fin" << endl;
    test_contadores();
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

5. Problema de los fumadores

```
#include <mutex>
#include <random> // dispositivos, generadores y distribuciones aleatorias
#include <chrono> // duraciones (duration), unidades de tiempo
#include "scd.h"

using namespace std;
using namespace scd;

const int num_fumadores = 3; // número de fumadores
Semaphore ingr_disp[num_fumadores] = {0,0,0};
Semaphore mostr_vacio = 1;

//-----
// Función que simula la acción de producir un ingrediente, como un retardo
// aleatorio de la hebra (devuelve número de ingrediente producido)

int producir_ingrediente(){
    // calcular milisegundos aleatorios de duración de la acción de fumar
    chrono::milliseconds duracion_produ( aleatorio<10,100>() );

    // informa de que comienza a producir
    cout << "Estanquero: empieza a producir ingrediente (" << duracion_produ.count() << "
    milisegundos)" << endl;

    // espera bloqueada un tiempo igual a 'duracion_produ' milisegundos
    this_thread::sleep_for(duracion_produ);

    const int num_ingrediente = aleatorio<0,num_fumadores-1>();

    // informa de que ha terminado de producir
    cout << "Estanquero: termina de producir ingrediente " << num_ingrediente << endl;

    return num_ingrediente;
}

//-----
// función que ejecuta la hebra del estanquero
void funcion_hebra_estanquero(){
    while(true){
        int ingrediente = producir_ingrediente();
        sem_wait(mostr_vacio);
        cout << "Estanquero: coloca ingrediente " << ingrediente << " en el mostrador" <<
        endl;
        sem_signal(ingr_disp[ingrediente]);
    }
}
```



```
//-----
// Función que simula la acción de fumar, como un retardo aleatoria de la hebra

void fumar(int num_fumador){
    // calcular milisegundos aleatorios de duración de la acción de fumar)
    chrono::milliseconds duracion_fumar( aleatorio<20,200>() );

    // informa de que comienza a fumar
    cout << "Fumador " << num_fumador << ":"
    << " empieza a fumar (" << duracion_fumar.count() << " milisegundos)" << endl;

    // espera bloqueada un tiempo igual a "duracion_fumar' milisegundos
    this_thread::sleep_for(duracion_fumar);

    // informa de que ha terminado de fumar
    cout << "Fumador " << num_fumador << ": termina de fumar, comienza espera de
    ingrediente." << endl;
}

//-----
// función que ejecuta la hebra del fumador
void funcion_hebra_fumador(int num_fumador){
    while(true){
        sem_wait(ingr_disp[num_fumador]);
        cout << "Fumador " << num_fumador << ": retira ingrediente del mostrador" << endl;
        sem_signal(mostr_vacio);
        fumar(num_fumador);
    }
}

//-----

int main(){
    // declarar hebras y ponerlas en marcha
    thread hebra_estanquero, hebra_fumador[num_fumadores];
    hebra_estanquero = thread(funcion_hebra_estanquero);
    for (int i=0; i<num_fumadores; i++){
        hebra_fumador[i] = thread(funcion_hebra_fumador, i);
    }

    hebra_estanquero.join();
    for (int i=0; i<num_fumadores; i++){
        hebra_fumador[i].join();
    }
    return 0;
}

```

Seminario 2. Introducción a los monitores en C++11.

1. Actividad: *monitor_em.cpp*.

```
#include <iostream>
#include <cassert>
#include <thread>
#include <mutex>
#include <random>
#include <chrono>
#include "scd.h"

using namespace std ;
using namespace scd ;
using namespace std::chrono;

const int num_incrementos = 10000 ;

// *****
// clase contador, sin exclusión mutua
class MContador1
{
private:
    int cont ;

public:
    MContador1( int valor_ini ) ;
    void incrementa();
    int leer_valor();
};

// -----
MContador1::MContador1( int valor_ini )
{
    cont = valor_ini ;
}
// -----
void MContador1::incrementa()
{
    cont ++ ;
}
// -----
int MContador1::leer_valor()
{
    return cont ;
}
```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
// *****
// clase contador, con exclusión mutua mediante herencia de 'HoareMonitor'
class MContador2 : public HoareMonitor
{
    private:
        int cont ;

    public:
        MContador2( int valor_ini ) ;
        void incrementa();
        int leer_valor();
};

// -----
MContador2::MContador2( int valor_ini )
{
    cont = valor_ini ;
}
// -----
void MContador2::incrementa()
{
    cont ++ ;
}
// -----
int MContador2::leer_valor()
{
    return cont ;
}

// *****
void funcion_hebra_M1( MContador1 & monitor )
{
    for( int i = 0 ; i < num_incrementos ; i++ )
        monitor.incrementa();
}
// -----
void test_1()
{
    MContador1 monitor(0) ;

    time_point<steady_clock> inicio = steady_clock::now();

    thread hebra1( funcion_hebra_M1, ref(monitor) ),
             hebra2( funcion_hebra_M1, ref(monitor) );

    hebra1.join();
    hebra2.join();
    time_point<steady_clock> fin = steady_clock::now();
}
```

```

duration<float,milli> tiempo = fin - inicio;
cout << "Monitor contador (sin exclusión mutua):" << endl
    << endl
    << " valor esperado == " << 2*num_incrementos << endl
    << " valor obtenido == " << monitor.leer_valor() << endl
    << " tiempo = " << tiempo.count() << " milisegundos " << endl
    << endl;
}
// *****

void funcion_hebra_M2( MRef<MContador2> monitor )
{
    for( int i = 0 ; i < num_incrementos ; i++ )
        monitor->incrementa();
}
// -----

void test_2()
{
    MRef<MContador2> monitor = Create<MContador2>(0) ;
    time_point<steady_clock> inicio = steady_clock::now();
    thread hebra1( funcion_hebra_M2, monitor ),
        hebra2( funcion_hebra_M2, monitor );

    hebra1.join();
    hebra2.join();
    time_point<steady_clock> fin = steady_clock::now();
    duration<float,milli> tiempo = fin - inicio;
    cout << "Monitor contador (EM usando clase derivada de HoareMonitor):" << endl
        << endl
        << " valor esperado == " << 2*num_incrementos << endl
        << " valor obtenido == " << monitor->leer_valor() << endl
        << " tiempo = " << tiempo.count() << " milisegundos " << endl
        << endl ;
}
// *****

int main()
{
    test_1();
    test_2();
}

```

```

teresa@teresa-VirtualBox: ~/Desktop/SCD-repaso/material/2a/scd-s2-fuente$ g++ -std=c++11 -pthread -o monitor_en monitor_en.cpp scd.cpp
teresa@teresa-VirtualBox: ~/Desktop/SCD-repaso/material/2a/scd-s2-fuente$ ./monitor_en
Monitor contador (sin exclusión mutua):
valor esperado == 20000
valor obtenido == 15583
tiempo = 0.567982 milisegundos

Monitor contador (EM usando clase derivada de HoareMonitor):
valor esperado == 20000
valor obtenido == 20000
tiempo = 2091.56 milisegundos

```

2. Problema del productor-consumidor (buffer FIFO)

```
#include <iostream>
#include <iomanip>
#include <cassert>
#include <random>
#include <thread>
#include "scd.h"

using namespace std;
using namespace scd;

constexpr int
    num_items = 15; // número de items a producir/consumir
int
    siguiente_dato = 0; // siguiente valor a devolver en 'producir_dato'

constexpr int
    min_ms     = 5,    // tiempo mínimo de espera en sleep_for
    max_ms     = 20;   // tiempo máximo de espera en sleep_for

mutex
    mtx;           // mutex de escritura en pantalla
unsigned
    cont_prod[num_items] = {0}, // contadores de verificación: producidos
    cont_cons[num_items] = {0}; // contadores de verificación: consumidos

//*****
// funciones comunes a las dos soluciones (fifo y lifo)
//-----

int producir_dato(){
    this_thread::sleep_for( chrono::milliseconds( aleatorio<min_ms,max_ms>() ));
    const int valor_producido = siguiente_dato;
    siguiente_dato++;
    mtx.lock();
    cout << "hebra productora, produce " << valor_producido << endl << flush ;
    mtx.unlock();
    cont_prod[valor_producido]++;
    return valor_producido;
}
```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
//-----

void consumir_dato(unsigned valor_consumir){
    if (num_items <= valor_consumir){
        cout << " valor a consumir === " << valor_consumir << ", num_items == " <<
num_items << endl ;
        assert(valor_consumir < num_items);
    }
    cont_cons[valor_consumir]++;
    this_thread::sleep_for( chrono::milliseconds( aleatorio<min_ms,max_ms>() ));
    mtx.lock();
    cout << "          hebra consumidora, consume: " << valor_consumir << endl ;
    mtx.unlock();
}

//-----

void test_contadores(){
    bool ok = true;
    cout << "comprobando contadores ...." << endl ;

    for(unsigned i = 0; i < num_items; i++){
        if (cont_prod[i] != 1){
            cout << "error: valor " << i << " producido " << cont_prod[i] << " veces." <<
endl;
            ok = false;
        }
        if (cont_cons[i] != 1){
            cout << "error: valor " << i << " consumido " << cont_cons[i] << " veces" <<
endl;
            ok = false;
        }
    }

    if (ok)
        cout << endl << flush << "solución (aparentemente) correcta." << endl << flush;
}
```



```
// *****
// clase para monitor buffer, version FIFO, semántica SC, multiples prod/cons

class ProdConsSU1 : public HoareMonitor{
private:
    static const int      // constantes ('static' ya que no dependen de la instancia)
        num_celdas_total = 10; // núm. de entradas del buffer

    int                  // variables permanentes
        buffer[num_celdas_total], // buffer de tamaño fijo, con los datos
        primera_libre,           // índice de celda de la próxima inserción
        primera_ocupada,         // índice de celda de la próxima extracción
        n;                       // número de celdas ocupadas

    CondVar              // colas condicion:
        ocupadas,          // cola donde espera el consumidor (n>0)
        libres;            // cola donde espera el productor (n<num_celdas_total)

public:                  // constructor y métodos públicos
    ProdConsSU1();        // constructor
    int leer();           // extraer un valor (sentencia L) (consumidor)
    void escribir(int valor); // insertar un valor (sentencia E) (productor)
};
// -----

ProdConsSU1::ProdConsSU1(){
    primera_libre = 0;
    primera_ocupada = 0;
    n = 0;
    ocupadas = newCondVar();
    libres = newCondVar();
}
// -----
// función llamada por el consumidor para extraer un dato

int ProdConsSU1::leer(){
    // esperar bloqueado hasta que 0 < primera_libre
    if (n == 0)
        ocupadas.wait();

    cout << "leer: ocup == " << n << ", total == " << num_celdas_total << endl;
    assert(0 < n);

    // hacer la operación de lectura, actualizando estado del monitor
    const int valor = buffer[primera_ocupada];
    primera_ocupada = (primera_ocupada + 1) % num_celdas_total;
    n--;
}
```

```

// señalar al productor que hay un hueco libre, por si está esperando
libres.signal();

// devolver valor
return valor;
}
// -----

void ProdConsSU1::escribir(int valor){
    // esperar bloqueado hasta que primera_libre < num_celdas_total
    if (n == num_celdas_total)
        libres.wait();

    cout << "escribir: ocup == " << n << ", total == " << num_celdas_total << endl;
    assert(n < num_celdas_total);

    // hacer la operación de inserción, actualizando estado del monitor
    buffer[primera_libre] = valor;
    primera_libre = (primera_libre + 1) % num_celdas_total;
    n++;

    // señalar al consumidor que ya hay una celda ocupada (por si esta esperando)
    ocupadas.signal();
}

// *****
// funciones de hebras

void funcion_hebra_productora(MRef<ProdConsSU1> monitor){
    for(unsigned i = 0; i < num_items; i++){
        int valor = producir_dato();
        monitor->escribir(valor);
    }
}
// -----

void funcion_hebra_consumidora(MRef<ProdConsSU1> monitor){
    for(unsigned i = 0; i < num_items; i++){
        int valor = monitor->leer();
        consumir_dato(valor);
    }
}
// -----

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



```
int main(){
    cout << "-----" << endl
    << "Problema del productor-consumidor únicos (Monitor SU, buffer FIFO). " << endl
    << "-----" << endl
    << flush;

    // crear monitor ('monitor' es una referencia al mismo, de tipo MRef<...>)
    MRef<ProdConsSU1> monitor = Create<ProdConsSU1>();

    // crear y lanzar las hebras
    thread hebra_prod(funcion_hebra_productora, monitor),
    hebra_cons(funcion_hebra_consumidora, monitor);

    // esperar a que terminen las hebras
    hebra_prod.join();
    hebra_cons.join();

    test_contadores();
}
```

Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

Práctica 2. Casos prácticos de monitores en C++11.

1. Problema de los múltiples productores-consumidores (buffer LIFO, monitor SU)

```
#include <iostream>
#include <iomanip>
#include <cassert>
#include <random>
#include <thread>
#include "scd.h"

using namespace std ;
using namespace scd ;

constexpr int
    num_items = 40, // número de items a producir/consumir
    np = 5, // número de productores
    nc = 4; // número de consumidores
int
    cont_datos[np] = {0},
    p = num_items/np, // número de items que produce cada productor
    c = num_items/nc; // número de items que consume cada consumidor

constexpr int
    min_ms = 5, // tiempo mínimo de espera en sleep_for
    max_ms = 20; // tiempo máximo de espera en sleep_for

mutex
    mtx; // mutex de escritura en pantalla
unsigned
    cont_prod[num_items] = {0}, // contadores de verificación: producidos
    cont_cons[num_items] = {0}; // contadores de verificación: consumidos

//*****
// funciones comunes a las dos soluciones (fifo y lifo)
//-----
int producir_dato(int ih){
    assert(ih < np);
    this_thread::sleep_for( chrono::milliseconds( aleatorio<min_ms,max_ms>() ));
    const int valor_producido = cont_datos[ih] + ih*p;
    cont_datos[ih]++;
    mtx.lock();
    cout << "+hebra productora " << ih << ", produce " << valor_producido << endl << flush;
    mtx.unlock();
    cont_prod[valor_producido]++;
    return valor_producido;
}
```

```
//-----

void consumir_dato(int ih, unsigned valor_consumir){
    assert(ih < nc);
    if (num_items <= valor_consumir){
        cout << "valor a consumir == " << valor_consumir << ", num_items == " <<
num_items << endl ;
        assert(valor_consumir < num_items);
    }
    cont_cons[valor_consumir]++;
    this_thread::sleep_for( chrono::milliseconds( aleatorio<min_ms,max_ms>() ));
    mtx.lock();
    cout << "-hebra consumidora " << ih << ", consume: " << valor_consumir << endl << flush;
    mtx.unlock();
}
//-----

void test_contadores(){
    bool ok = true;
    cout << "comprobando contadores ...." << endl ;

    for(unsigned i = 0; i < num_items; i++){
        if (cont_prod[i] != 1){
            cout << "error: valor " << i << " producido " << cont_prod[i] << " veces." <<
endl;
            ok = false;
        }
        if (cont_cons[i] != 1){
            cout << "error: valor " << i << " consumido " << cont_cons[i] << " veces" <<
endl;
            ok = false;
        }
    }

    if (ok)
        cout << endl << flush << "solución (aparentemente) correcta." << endl << flush;
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH



```
// *****
// clase para monitor buffer, version LIFO, semántica SU, multiples prod/cons

class ProdConsMu : public HoareMonitor{
private:
    static const int    // constantes ('static' ya que no dependen de la instancia)
        num_celdas_total = 10; // núm. de entradas del buffer

    int                // variables permanentes
        buffer[num_celdas_total], // buffer de tamaño fijo, con los datos
        primera_libre;           // índice de celda de la próxima inserción ( == número de
                                // celdas ocupadas)

    CondVar            // colas condicion:
        ocupadas,      // cola donde esperan los consumidores (n>0)
        libres;         // cola donde esperan los productores (n<num_celdas_total)

public:                // constructor y métodos públicos
    ProdConsMu();      // constructor
    int leer();         // extraer un valor
    void escribir(int valor); // insertar un valor
};
// -----
ProdConsMu::ProdConsMu(){
    primera_libre = 0;
    ocupadas = newCondVar();
    libres = newCondVar();
}
// -----
// función llamada por el consumidor para extraer un dato
int ProdConsMu::leer(){
    // esperar bloqueado hasta que 0 < primera_libre
    if (primera_libre == 0)
        ocupadas.wait();

    cout << "                                leer: ocup == " << primera_libre << ", total == " <<
num_celdas_total << endl;
    assert(0 < primera_libre);

    // hacer la operación de lectura, actualizando estado del monitor
    primera_libre--;
    const int valor = buffer[primera_libre];

    // señalar al productor que hay un hueco libre, por si está esperando
    libres.signal();

    return valor;
}
```

```
// -----

void ProdConsMu::escribir(int valor){
    // esperar bloqueado hasta que primera_libre < num_celdas_total
    if (primera_libre == num_celdas_total)
        libres.wait();

    cout << "                                escribir: ocup == " << primera_libre << ", total == " <<
num_celdas_total << endl;
    assert(primera_libre < num_celdas_total);

    // hacer la operación de inserción, actualizando estado del monitor
    buffer[primera_libre] = valor;
    primera_libre++;

    // señalar al consumidor que ya hay una celda ocupada (por si esta esperando)
    ocupadas.signal();
}

// *****
// funciones de hebras

void funcion_hebra_productora(int ih, MRef<ProdConsMu> monitor){
    for(unsigned i=0; i<p; i++){
        int valor = producir_dato(ih);
        monitor->escribir(valor);
    }
}

// -----

void funcion_hebra_consumidora(int ih, MRef<ProdConsMu> monitor){
    for(unsigned i=0; i<c; i++){
        int valor = monitor->leer();
        consumir_dato(ih, valor);
    }
}

// -----
```



```

int main(){
    cout << "-----" << endl
    << "Problema de los múltiples productores y consumidores (Monitor SU, buffer LIFO). "
<< endl
    << "-----" << endl
    << flush;

    // crear monitor ('monitor' es una referencia al mismo, de tipo MRef<...>)
    MRef<ProdConsMu> monitor = Create<ProdConsMu>();

    // crear y lanzar las hebras
    thread hebras_prod[np], hebras_cons[nc];

    for (int i=0; i<np; i++){
        hebras_prod[i] = thread(funcion_hebra_productora, i, monitor);
    }
    for (int i=0; i<nc; i++){
        hebras_cons[i] = thread(funcion_hebra_consumidora, i, monitor);
    }

    // esperar a que terminen las hebras
    for (int i=0; i<np; i++){
        hebras_prod[i].join();
    }
    for (int i=0; i<nc; i++){
        hebras_cons[i].join();
    }

    test_contadores();
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

2. Problema de los múltiples productores-consumidores (buffer FIFO, monitor SU)

```
#include <iostream>
#include <iomanip>
#include <cassert>
#include <random>
#include <thread>
#include "scd.h"

using namespace std ;
using namespace scd ;

constexpr int
    num_items = 40, // número de items a producir/consumir
    np = 5, // número de productores
    nc = 4; // número de consumidores

int
    cont_datos[np] = {0},
    p = num_items/np, // número de items que produce cada productor
    c = num_items/nc; // número de items que consume cada consumidor

constexpr int
    min_ms = 5, // tiempo mínimo de espera en sleep_for
    max_ms = 20; // tiempo máximo de espera en sleep_for

mutex
    mtx; // mutex de escritura en pantalla

unsigned
    cont_prod[num_items] = {0}, // contadores de verificación: producidos
    cont_cons[num_items] = {0}; // contadores de verificación: consumidos

//*****
// funciones comunes a las dos soluciones (fifo y lifo)
//-----

int producir_dato(int ih){
    assert(ih < np);
    this_thread::sleep_for( chrono::milliseconds( aleatorio<min_ms,max_ms>() ));
    const int valor_producido = cont_datos[ih] + ih*p;
    cont_datos[ih]++;
    mtx.lock();
    cout << "+hebra productora " << ih << ", produce " << valor_producido << endl << flush;
    mtx.unlock();
    cont_prod[valor_producido]++;
    return valor_producido;
}
```

```
//-----

void consumir_dato(int ih, unsigned valor_consumir){
    assert(ih < nc);
    if (num_items <= valor_consumir){
        cout << "valor a consumir == " << valor_consumir << ", num_items == " <<
num_items << endl ;
        assert(valor_consumir < num_items);
    }
    cont_cons[valor_consumir]++;
    this_thread::sleep_for( chrono::milliseconds( aleatorio<min_ms,max_ms>() ));
    mtx.lock();
    cout << "-hebra consumidora " << ih << ", consume: " << valor_consumir << endl << flush;
    mtx.unlock();
}

//-----

void test_contadores(){
    bool ok = true;
    cout << "comprobando contadores ...." << endl ;

    for(unsigned i = 0; i < num_items; i++){
        if (cont_prod[i] != 1){
            cout << "error: valor " << i << " producido " << cont_prod[i] << " veces." <<
endl;
            ok = false;
        }
        if (cont_cons[i] != 1){
            cout << "error: valor " << i << " consumido " << cont_cons[i] << " veces" <<
endl;
            ok = false;
        }
    }

    if (ok)
        cout << endl << flush << "solución (aparentemente) correcta." << endl << flush;
}

// *****
```

```

// clase para monitor buffer, version FIFO, semántica SU, multiples prod/cons

class ProdConsMu : public HoareMonitor{
private:
    static const int      // constantes ('static' ya que no dependen de la instancia)
        num_celdas_total = 10; // núm. de entradas del buffer

    int                  // variables permanentes
        buffer[num_celdas_total], // buffer de tamaño fijo, con los datos
        primera_libre,           // índice de celda de la próxima inserción
        primera_ocupada,         // índice de celda de la próxima extracción
        n;                       // número de celdas ocupadas

    CondVar              // colas condicion:
        ocupadas,           // cola donde esperan los consumidores (n>0)
        libres;             // cola donde esperan los productores (n<num_celdas_total)

public:                  // constructor y métodos públicos
    ProdConsMu();         // constructor
    int leer();           // extraer un valor
    void escribir(int valor); // insertar un valor
};
// -----
ProdConsMu::ProdConsMu(){
    primera_libre = 0;
    primera_ocupada = 0;
    n = 0;
    ocupadas = newCondVar();
    libres = newCondVar();
}
// -----
// función llamada por el consumidor para extraer un dato

int ProdConsMu::leer(){
    // esperar bloqueado hasta que 0 < primera_libre
    if (n == 0)
        ocupadas.wait();

    mtx.lock();
    cout << "                      leer: ocup == " << n << ", total == " <<
num_celdas_total << endl;
    mtx.unlock();
    assert(0 < n);

    // hacer la operación de lectura, actualizando estado del monitor
    const int valor = buffer[primera_ocupada];
    primera_ocupada = (primera_ocupada + 1) % num_celdas_total;
    n--;
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali oohh
esto con 1 coin me
lo quito yo...

WUOLAH

```
// señalar al productor que hay un hueco libre, por si está esperando
libres.signal();

// devolver valor
return valor;
}
// -----

void ProdConsMu::escribir(int valor){
    // esperar bloqueado hasta que primera_libre < num_celdas_total
    if (n == num_celdas_total)
        libres.wait();

    mtx.lock();
    cout << "                escribir: ocup == " << n << ", total == " <<
num_celdas_total << endl;
    mtx.unlock();
    assert(n < num_celdas_total);

    // hacer la operación de inserción, actualizando estado del monitor
    buffer[primera_libre] = valor;
    primera_libre = (primera_libre + 1) % num_celdas_total;
    n++;

    // señalar al consumidor que ya hay una celda ocupada (por si esta esperando)
    ocupadas.signal();
}

// *****
// funciones de hebras

void funcion_hebra_productora(int ih, MRef<ProdConsMu> monitor){
    for(unsigned i=0; i<p; i++){
        int valor = producir_dato(ih);
        monitor->escribir(valor);
    }
}
// -----

void funcion_hebra_consumidora(int ih, MRef<ProdConsMu> monitor){
    for(unsigned i=0; i<c; i++){
        int valor = monitor->leer();
        consumir_dato(ih, valor);
    }
}
// -----
```

```

int main(){
    cout << "-----" << endl
    << "Problema de los múltiples productores y consumidores (Monitor SU, buffer FIFO). "
<< endl
    << "-----" << endl
    << flush;

    // crear monitor ('monitor' es una referencia al mismo, de tipo MRef<...>)
    MRef<ProdConsMu> monitor = Create<ProdConsMu>();

    // crear y lanzar las hebras
    thread hebras_prod[np], hebras_cons[nc];

    for (int i=0; i<np; i++){
        hebras_prod[i] = thread(funcion_hebra_productora, i, monitor);
    }
    for (int i=0; i<nc; i++){
        hebras_cons[i] = thread(funcion_hebra_consumidora, i, monitor);
    }

    // esperar a que terminen las hebras
    for (int i=0; i<np; i++){
        hebras_prod[i].join();
    }
    for (int i=0; i<nc; i++){
        hebras_cons[i].join();
    }

    test_contadores();
}

```

3. Problema de los fumadores (monitor SU)

```
#include <iostream>
#include <iomanip>
#include <cassert>
#include <random>
#include <thread>
#include "scd.h"

using namespace std;
using namespace scd;

const int num_fumadores = 3; // numero de fumadores

//-----
// Función que simula la acción de producir un ingrediente, como un retardo
// aleatorio de la hebra (devuelve número de ingrediente producido)

int producir_ingrediente(){
    // calcular milisegundos aleatorios de duración de la acción de fumar)
    chrono::milliseconds duracion_produ( aleatorio<10,100>() );

    // informa de que comienza a producir
    cout << "Estanquero: empieza a producir ingrediente (" << duracion_produ.count() << "
milisegundos)" << endl;

    // espera bloqueada un tiempo igual a "duracion_produ" milisegundos
    this_thread::sleep_for(duracion_produ);

    const int num_ingrediente = aleatorio<0,num_fumadores-1>();

    // informa de que ha terminado de producir
    cout << "Estanquero: termina de producir ingrediente " << num_ingrediente << endl;

    return num_ingrediente;
}

//-----
```


Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

// Función que simula la acción de fumar, como un retardo aleatoria de la hebra

```
void fumar(int num_fumador){
    // calcular milisegundos aleatorios de duración de la acción de fumar)
    chrono::milliseconds duracion_fumar( aleatorio<20,200>() );

    // informa de que comienza a fumar
    cout << "Fumador " << num_fumador << ":"
    << " empieza a fumar (" << duracion_fumar.count() << " milisegundos)" << endl;

    // espera bloqueada un tiempo igual a "duracion_fumar" milisegundos
    this_thread::sleep_for(duracion_fumar);

    // informa de que ha terminado de fumar
    cout << "Fumador " << num_fumador << ": termina de fumar, comienza espera de
    ingrediente." << endl;
}
```

// *****
// clase para monitor buffer, version FIFO, semántica SU, multiples prod/cons

```
class Estanco : public HoareMonitor{
private:
    int                // variables permanentes
        mostrador;

    CondVar            // colas condicion:
        ingr_disp[num_fumadores],          // cola donde esperan los fumadores
        mostr_libre;                        // cola donde espera el estanquero

public:
    // constructor y métodos públicos
    Estanco();                          // constructor
    void obtenerIngrediente(int ingrediente);
    void ponerIngrediente(int ingrediente);
    void esperarRecogidaIngrediente();
};
```

```
// -----
Estanco::Estanco(){
    mostrador = -1;
    for (int i=0; i<num_fumadores; i++){
        ingr_disp[i] = newCondVar();
    }
    mostr_libre = newCondVar();
}
// -----
```

```

void Estanco::obtenerIngrediente(int ingrediente){
    if (mostrador != ingrediente){
        ingr_disp[ingrediente].wait();
    }
    mostrador = -1;
    cout << "Fumador " << ingrediente << ": retira ingrediente del mostrador" << endl;
    mostr_libre.signal();
}
//-----

void Estanco::ponerIngrediente(int ingrediente){
    if (mostrador != -1){
        mostr_libre.wait();
    }
    mostrador = ingrediente;
    cout << "Estanquero: coloca ingrediente " << ingrediente << " en el mostrador" << endl;
    ingr_disp[ingrediente].signal();
}
//-----

void Estanco::esperarRecogidaIngrediente(){
    if (mostrador!=-1){
        mostr_libre.wait();
    }
}

//-----
// función que ejecuta la hebra del fumador
void funcion_hebra_fumador(int num_fumador, MRef<Estanco> monitor){
    while(true){
        monitor->obtenerIngrediente(num_fumador);
        fumar(num_fumador);
    }
}
//-----
// función que ejecuta la hebra del estanquero

void funcion_hebra_estanquero(MRef<Estanco> monitor){
    while(true){
        int ingrediente = producir_ingrediente();
        monitor->ponerIngrediente(ingrediente);
        monitor->esperarRecogidaIngrediente();
    }
}

```

```
//-----
int main(){
    cout << "-----" << endl
    << "Problema de los fumadores (Monitor SU). " << endl
    << "-----" << endl
    << flush;

    // crear monitor ('monitor' es una referencia al mismo, de tipo MRef<...>)
    MRef<Estanco> monitor = Create<Estanco>();

    // crear y lanzar las hebras
    thread hebra_estanquero, hebra_fumador[num_fumadores];
    hebra_estanquero = thread(funcion_hebra_estanquero, monitor);
    for (int i=0; i<num_fumadores; i++){
        hebra_fumador[i] = thread(funcion_hebra_fumador, i, monitor);
    }

    // esperar a que terminen las hebras
    hebra_estanquero.join();
    for (int i=0; i<num_fumadores; i++){
        hebra_fumador[i].join();
    }

    return 0;
}
```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

4. Problema de los lectores y escritores (monitor SU, prioridad lectores)

Se ha dado prioridad a los lectores (en el momento que un escritor termina, si hay escritores y lectores esperando, pasan los lectores).

```
#include <iostream>
#include <iomanip>
#include <cassert>
#include <random>
#include <thread>
#include "scd.h"

using namespace std;
using namespace scd;

const int
    nl = 2, // número total de lectores
    ne = 3; // número total de escritores

// *****
// clase para monitor LecEsc, semántica SU

class LecEsc : public HoareMonitor{
private:
    bool escrib; // true si hay escritores escribiendo
    int n_lec; // número de lectores leyendo
    CondVar // variables condición
        lectura, // cola donde esperan los lectores cuando escrib==true
        escritura; // cola donde esperan los escritores cuando escrib==true o n_lec>0

public:
    LecEsc();
    void ini_lectura();
    void fin_lectura();
    void ini_escritura();
    void fin_escritura();
};

//-----
LecEsc::LecEsc(){
    escrib = false;
    n_lec = 0;
    lectura = newCondVar();
    escritura = newCondVar();
}

//-----
```

```

void LecEsc::ini_lectura(){
    // si hay alguien escribiendo, esperar
    if (escrib)
        lectura.wait();
    n_lect++; // registrar que hay un lector más
    cout << "+Lector comienza a leer, número de lectores: " << n_lect << endl;
    lectura.signal(); // desbloquear en cadena posibles lectores bloqueados
}

//-----
void LecEsc::fin_lectura(){
    n_lect--; // registrar que hay un lector menos
    cout << "-Lector termina de leer, número de lectores: " << n_lect << endl;
    if (n_lect == 0){ // si es el último lector
        escritura.signal(); // desbloquear escritor
    }
}

//-----
void LecEsc::ini_escritura(){
    // si hay otros, esperar
    if (escrib || n_lect>0)
        escritura.wait();

    cout << "                                +Escritor comienza a escribir" << endl;
    // registrar que hay un escritor
    escrib = true;
}

//-----
void LecEsc::fin_escritura(){
    escrib = false; // registrar que ya no hay escritor
    cout << "                                -Escritor termina de escribir" << endl;
    if (!lectura.empty()){ // si hay lectores esperando
        lectura.signal(); // desbloquear a uno
    }
    else{ // si no
        escritura.signal(); // desbloquear a un escritor
    }
}

//-----
// función que ejecuta las hebras lectoras
void funcion_hebra_lectora(MRef<LecEsc> monitor){
    while (true) {
        monitor->ini_lectura(); // inicio lectura
        this_thread::sleep_for((chrono::milliseconds) aleatorio<100,250>() ); // retraso
        aleatorio simulando lectura
    }
}

```

```

        monitor->fin_lectura(); // fin lectura
        this_thread::sleep_for((chrono::milliseconds) aleatorio<30,100>() ); // retraso
aleatorio
    }
}

//-----
// función que ejecuta las hebras escritoras
void funcion_hebra_escritora(MRef<LecEsc> monitor){
    while (true) {
        monitor->ini_escritura(); // inicio escritura
        this_thread::sleep_for((chrono::milliseconds) aleatorio<100,250>() ); // retraso
aleatorio simulando escritura
        monitor->fin_escritura(); // fin escritura
        this_thread::sleep_for((chrono::milliseconds) aleatorio<30,100>() ); // retraso
aleatorio
    }
}

//-----
int main(){
    cout << "-----" << endl
    << "Problema de los lectores y escritores (prioridad para lectores, Monitor SU). " << endl
    << "-----" << endl
    << "Número total de lectores: " << nl << endl
    << "Número total de escritores: " << ne << endl << flush;

    // crear monitor
    MRef<LecEsc> monitor = Create<LecEsc>();

    // crear y lanzar las hebras
    thread hebra_lector[nl], hebra_escritor[ne];
    for (int i=0; i<nl; i++){
        hebra_lector[i] = thread(funcion_hebra_lectora, monitor);
    }
    for (int i=0; i<ne; i++){
        hebra_escritor[i] = thread(funcion_hebra_escritora, monitor);
    }

    // esperar a que terminen las hebras
    for (int i=0; i<nl; i++){
        hebra_lector[i].join();
    }
    for (int i=0; i<ne; i++){
        hebra_escritor[i].join();
    }
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

perdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

Práctica 3. Implementación de algoritmos distribuidos con MPI

1. Problema de los múltiples productores y consumidores (buffer FIFO)

```
#include <iostream>
#include <thread> // this_thread::sleep_for
#include <random> // dispositivos, generadores y distribuciones aleatorias
#include <chrono> // duraciones (duration), unidades de tiempo
#include <mpi.h>

using namespace std;
using namespace std::this_thread;
using namespace std::chrono;

const int
    np = 4, // número de procesos productores
    nc = 5, // número de procesos consumidores
    m = 20, // número de ítems a producir o consumir
    tam_vector = 10, // tamaño del buffer
    etiq_prod = 1, // etiqueta para mensajes de los productores
    etiq_cons = 2; // etiqueta para mensajes de los consumidores

int
    id_productor_min = 0, // id del primer productor
    id_productor_max = np - 1, // id del último productor
    id_buffer = np, // id del proceso buffer
    id_consumidor_min = np + 1, // id del primer consumidor
    id_consumidor_max = np + nc, // id del último consumidor
    num_procesos_esperado = np + nc + 1, // número de procesos esperado (productores +
    consumidores + buffer)
    p = m/np, // número de ítems que produce cada productor
    c = m/nc; // número de ítems que consume cada consumidor

//*****
// plantilla de función para generar un entero aleatorio uniformemente
// distribuido entre dos valores enteros, ambos incluidos
// (ambos tienen que ser dos constantes, conocidas en tiempo de compilación)
//-----

template< int min, int max > int aleatorio(){
    static default_random_engine generador( (random_device())() );
    static uniform_int_distribution<int> distribucion_uniforme( min, max );
    return distribucion_uniforme(generador);
}
// -----
```



```

// producir produce los numeros en secuencia (1,2,3,...) y lleva espera aleatorio

int producir(int orden){
    static int contador[np] = {0};
    sleep_for( milliseconds( aleatorio<10,100>()) );
    contador[orden]++;
    cout << "Productor " << orden << " ha producido valor " << contador[orden] << endl <<
flush;
    return contador[orden];
}

// -----

void funcion_productor(int orden){
    for (unsigned int i=0; i<p; i++){
        // producir valor
        int valor_prod = producir(orden);
        // enviar valor
        cout << "Productor " << orden << " va a enviar valor " << valor_prod << endl <<
flush;
        MPI_Ssend(&valor_prod, 1, MPI_INT, id_buffer, etiq_prod, MPI_COMM_WORLD);
    }
}

// -----

void consumir(int orden, int valor_cons){
    // espera bloqueada
    sleep_for( milliseconds( aleatorio<110,200>()) );
    cout << "Consumidor " << orden << " ha consumido valor " << valor_cons << endl <<
flush;
}

// -----

void funcion_consumidor(int orden){
    int peticion, valor_rec = 1;
    MPI_Status estado;
    for(unsigned int i=0; i<c; i++){
        MPI_Ssend(&peticion, 1, MPI_INT, id_buffer, etiq_cons, MPI_COMM_WORLD);
        MPI_Recv (&valor_rec, 1, MPI_INT, id_buffer, etiq_cons, MPI_COMM_WORLD,
&estado);
        cout << "Consumidor " << orden << " ha recibido valor " << valor_rec << endl <<
flush;
        consumir(orden, valor_rec);
    }
}

// -----

```

```

void funcion_buffer(){
    int
        buffer[tam_vector], // buffer con celdas ocupadas y vacías
        valor,                // valor recibido o enviado
        primera_libre         = 0, // índice de primera celda libre
        primera_ocupada       = 0, // índice de primera celda ocupada
        num_celdas_ocupadas = 0, // número de celdas ocupadas
        etiq_aceptable;        // etiqueta de emisor aceptable

    MPI_Status estado;          // metadatos del mensaje recibido

    for(unsigned int i=0 ;i<m*2; i++){
        // 1. determinar si puede enviar solo prod., solo cons, o todos
        if (num_celdas_ocupadas == 0) // si buffer vacío
            etiq_aceptable = etiq_prod; // $$$$ solo prod.
        else if (num_celdas_ocupadas == tam_vector) // si buffer lleno
            etiq_aceptable = etiq_cons ; // $$$$ solo cons.
        else // si no vacío ni lleno
            etiq_aceptable = MPI_ANY_TAG; // $$$$ cualquiera

        // 2. recibir un mensaje del emisor o emisores aceptables
        MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_aceptable,
MPI_COMM_WORLD, &estado);

        // 3. procesar el mensaje recibido
        switch(estado.MPI_TAG){ // leer etiqueta del mensaje en metadatos
            case etiq_prod: // si ha sido el productor: insertar en buffer
                buffer[primera_libre] = valor;
                primera_libre = (primera_libre+1) % tam_vector;
                num_celdas_ocupadas++;
                cout << "Buffer ha recibido valor " << valor << endl;
                break;

            case etiq_cons: // si ha sido el consumidor: extraer y enviarle
                valor = buffer[primera_ocupada];
                primera_ocupada = (primera_ocupada+1) % tam_vector;
                num_celdas_ocupadas--;
                cout << "Buffer va a enviar valor " << valor << endl;
                MPI_Ssend(&valor, 1, MPI_INT, estado.MPI_SOURCE, etiq_cons,
MPI_COMM_WORLD);
                break;

        }
    }
}
// -----

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
int main(int argc, char *argv[]){
    int id_propio, num_procesos_actual;

    // inicializar MPI, leer identif. de proceso y número de procesos
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id_propio);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procesos_actual);

    if (num_procesos_esperado == num_procesos_actual){
        // ejecutar la operación apropiada a 'id_propio'
        if (id_propio >= id_productor_min && id_propio <= id_productor_max)
            funcion_productor(id_propio);

        else if (id_propio == id_buffer)
            funcion_buffer();

        else
            funcion_consumidor(id_propio - id_consumidor_min);
    }
    else{
        if (id_propio == 0){ // solo el primero escribe error, indep. del rol
            cout << "el número de procesos esperados es: " <<
num_procesos_esperado << endl
                << "el número de procesos en ejecución es: " <<
num_procesos_actual << endl
                << "(programa abortado)" << endl;
        }
    }

    // al terminar el proceso, finalizar MPI
    MPI_Finalize();
    return 0;
}
```

2. Cena de los filósofos (primera aproximación con interbloqueo)

Con esta implementación si todos los filósofos cogen su tenedor izquierdo antes de que ninguno pueda coger el derecho se quedarán todos los procesos filósofos bloqueados esperando a poder coger el tenedor derecho y ninguno podrá soltar el tenedor que tiene.

```
teresa@teresa-VirtualBox:~/Desktop/SCD-repaso/material/3b/scd-p3-fuentes$ mpirun -oversubscribe -np 10 filosofos-interb
Filósofo 2 solicita ten. izq.3
Filósofo 4 solicita ten. izq.5
Filósofo 0 solicita ten. izq.1
Ten. 1 ha sido cogido por filo. 0
Filósofo 8 solicita ten. izq.9
Ten. 3 ha sido cogido por filo. 2
Filósofo 2 solicita ten. der.1
Filósofo 0 solicita ten. der.9
Ten. 5 ha sido cogido por filo. 4
Filósofo 4 solicita ten. der.3
Filósofo 6 solicita ten. izq.7
Ten. 7 ha sido cogido por filo. 6
Filósofo 6 solicita ten. der.5
Filósofo 8 solicita ten. der.7
Ten. 9 ha sido cogido por filo. 8
```

```
#include <mpi.h>
#include <thread> // this_thread::sleep_for
#include <random> // dispositivos, generadores y distribuciones aleatorias
#include <chrono> // duraciones (duration), unidades de tiempo
#include <iostream>

using namespace std;
using namespace std::this_thread;
using namespace std::chrono;

const int
    num_filosofos = 5 ,           // número de filósofos
    num_filo_ten  = 2*num_filosofos, // número de filósofos y tenedores
    num_procesos  = num_filo_ten,   // número de procesos total (por ahora solo hay filo y
ten)
    etiq_filo_solicitar = 1,
    etiq_filo_soltar    = 2;

//*****
// plantilla de función para generar un entero aleatorio uniformemente
// distribuido entre dos valores enteros, ambos incluidos
// (ambos tienen que ser dos constantes, conocidas en tiempo de compilación)
//-----

template< int min, int max > int aleatorio(){
    static default_random_engine generador( (random_device())() );
    static uniform_int_distribution<int> distribucion_uniforme( min, max );
    return distribucion_uniforme( generador );
}
```

```
// -----

void funcion_filosofos(int id){
    int id_ten_izq = (id+1) % num_filo_ten; //id. tenedor izq.
    int id_ten_der = (id+num_filo_ten-1) % num_filo_ten; //id. tenedor der.
    int peticion;

    while (true){
        cout <<"Filósofo " <<id << " solicita ten. izq." <<id_ten_izq <<endl;
        // ... solicitar tenedor izquierdo
        MPI_Ssend(&peticion, 1, MPI_INT, id_ten_izq, etiq_filo_solicitar,
MPI_COMM_WORLD);

        cout <<"Filósofo " <<id <<" solicita ten. der." <<id_ten_der <<endl;
        // ... solicitar tenedor derecho
        MPI_Ssend(&peticion, 1, MPI_INT, id_ten_der, etiq_filo_solicitar,
MPI_COMM_WORLD);

        cout <<"Filósofo " <<id <<" comienza a comer" <<endl;
        sleep_for( milliseconds( aleatorio<10,100>() ) );

        cout <<"Filósofo " <<id <<" suelta ten. izq. " <<id_ten_izq <<endl;
        // ... soltar el tenedor izquierdo
        MPI_Ssend(&peticion, 1, MPI_INT, id_ten_izq, etiq_filo_soltar,
MPI_COMM_WORLD);

        cout<< "Filósofo " <<id <<" suelta ten. der. " <<id_ten_der <<endl;
        // ... soltar el tenedor derecho
        MPI_Ssend(&peticion, 1, MPI_INT, id_ten_der, etiq_filo_soltar,
MPI_COMM_WORLD);

        cout << "Filosofo " << id << " comienza a pensar" << endl;
        sleep_for( milliseconds( aleatorio<10,100>() ) );
    }
}

// -----
```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
void funcion_tenedores(int id){
    int valor, id_filosofo; // valor recibido, identificador del filósofo
    MPI_Status estado; // metadatos de las dos recepciones

    while (true){
        // ..... recibir petición de cualquier filósofo
        MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_filo_solicitar,
        MPI_COMM_WORLD, &estado);
        // ..... guardar en 'id_filosofo' el id. del emisor
        id_filosofo = estado.MPI_SOURCE;
        cout <<"Ten. " <<id <<" ha sido cogido por filo. " << id_filosofo <<endl;

        // ..... recibir liberación de filósofo 'id_filosofo'
        MPI_Recv(&valor, 1, MPI_INT, estado.MPI_SOURCE, etiq_filo_soltar,
        MPI_COMM_WORLD, &estado);
        cout <<"Ten. " <<id <<" ha sido liberado por filo. " << id_filosofo <<endl ;
    }
}

// -----

int main(int argc, char** argv){
    int id_propio, num_procesos_actual;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id_propio);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procesos_actual);

    if (num_procesos == num_procesos_actual){
        // ejecutar la función correspondiente a 'id_propio'
        if (id_propio % 2 == 0) // si es par
            funcion_filosofos(id_propio); // es un filósofo

        else // si es impar
            funcion_tenedores(id_propio); // es un tenedor
    }
    else{
        if (id_propio == 0){ // solo el primero escribe error, indep. del rol
            cout << "el número de procesos esperados es: " << num_procesos <<
endl
            << "el número de procesos en ejecución es: " << num_procesos_actual <<
endl
            << "(programa abortado)" << endl;
        }
    }

    MPI_Finalize();
    return 0;
}
```

3. Cena de los filósofos con camarero

```
#include <mpi.h>
#include <thread> // this_thread::sleep_for
#include <random> // dispositivos, generadores y distribuciones aleatorias
#include <chrono> // duraciones (duration), unidades de tiempo
#include <iostream>

using namespace std;
using namespace std::this_thread;
using namespace std::chrono;

const int
    num_filosofos = 5 ,           // número de filósofos
    num_filo_ten = 2*num_filosofos, // número de filósofos y tenedores
    num_procesos = num_filo_ten + 1, // número de procesos total
    id_camarero = num_filo_ten,    // id del camarero
    etiq_solicitar = 1,
    etiq_soltar = 2,
    etiq_sentarse = 3,
    etiq_levantarse = 4;

//*****
// plantilla de función para generar un entero aleatorio uniformemente
// distribuido entre dos valores enteros, ambos incluidos
// (ambos tienen que ser dos constantes, conocidas en tiempo de compilación)
//-----

template< int min, int max > int aleatorio(){
    static default_random_engine generador( (random_device())() );
    static uniform_int_distribution<int> distribucion_uniforme( min, max );
    return distribucion_uniforme( generador );
}

// -----

void funcion_filosofos(int id){
    int id_ten_izq = (id+1) % num_filo_ten; //id. tenedor izq.
    int id_ten_der = (id+num_filo_ten-1) % num_filo_ten; //id. tenedor der.
    int peticion;

    while (true){
        // ... solicitar sentarse
        cout <<"Filósofo " << id << " solicita sentarse" <<endl;
        MPI_Ssend(&peticion, 1, MPI_INT, id_camarero, etiq_sentarse,
MPI_COMM_WORLD);
```



```

    cout << "Filósofo " << id << " solicita ten. izq." << id_ten_izq << endl;
    // ... solicitar tenedor izquierdo
    MPI_Ssend(&peticion, 1, MPI_INT, id_ten_izq, etiq_solicitar, MPI_COMM_WORLD);

    cout << "Filósofo " << id << " solicita ten. der." << id_ten_der << endl;
    // ... solicitar tenedor derecho
    MPI_Ssend(&peticion, 1, MPI_INT, id_ten_der, etiq_solicitar, MPI_COMM_WORLD);

    cout << "Filósofo " << id << " comienza a comer" << endl;
    sleep_for( milliseconds( aleatorio<10,100>() ) );

    cout << "Filósofo " << id << " suelta ten. izq. " << id_ten_izq << endl;
    // ... soltar el tenedor izquierdo
    MPI_Ssend(&peticion, 1, MPI_INT, id_ten_izq, etiq_soltar, MPI_COMM_WORLD);

    cout << "Filósofo " << id << " suelta ten. der. " << id_ten_der << endl;
    // ... soltar el tenedor derecho
    MPI_Ssend(&peticion, 1, MPI_INT, id_ten_der, etiq_soltar, MPI_COMM_WORLD);

    // ... solicitar levantarse
    cout << "Filósofo " << id << " solicita levantarse" << endl;
    MPI_Ssend(&peticion, 1, MPI_INT, id_camarero, etiq_levantarse,
MPI_COMM_WORLD);

    cout << "Filosofo " << id << " comienza a pensar" << endl;
    sleep_for( milliseconds( aleatorio<10,100>() ) );
}
}
// -----

void funcion_tenedores(int id){
    int valor, id_filosofo; // valor recibido, identificador del filósofo
    MPI_Status estado; // metadatos de las dos recepciones

    while (true){
        // ..... recibir petición de cualquier filósofo
        MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_solicitar,
MPI_COMM_WORLD, &estado);
        // ..... guardar en 'id_filosofo' el id. del emisor
        id_filosofo = estado.MPI_SOURCE;
        cout << "Ten. " << id << " ha sido cogido por filo. " << id_filosofo << endl;

        // ..... recibir liberación de filósofo 'id_filosofo'
        MPI_Recv(&valor, 1, MPI_INT, estado.MPI_SOURCE, etiq_soltar,
MPI_COMM_WORLD, &estado);
        cout << "Ten. " << id << " ha sido liberado por filo. " << id_filosofo << endl ;
    }
}
}

```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

```
// -----
void funcion_camarero(){
    int valor, etiq_aceptable, sentados = 0;
    MPI_Status estado;

    while (true){
        if (sentados == num_filosofos-1)
            etiq_aceptable = etiq_levantarse;

        else if (sentados == 0)
            etiq_aceptable = etiq_sentarse;

        else
            etiq_aceptable = MPI_ANY_TAG;

        MPI_Recv(&valor, 1, MPI_INT, MPI_ANY_SOURCE, etiq_aceptable,
        MPI_COMM_WORLD, &estado);

        if (estado.MPI_TAG == etiq_sentarse){
            sentados++;
            cout << "Filósofo " << estado.MPI_SOURCE << " se ha sentado" << endl;
        }
        else if (estado.MPI_TAG == etiq_levantarse){
            sentados--;
            cout << "Filósofo " << estado.MPI_SOURCE << " se ha levantado" << endl;
        }
    }
}
// -----
int main(int argc, char** argv){
    int id_propio, num_procesos_actual;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &id_propio);
    MPI_Comm_size(MPI_COMM_WORLD, &num_procesos_actual);

    if (num_procesos == num_procesos_actual){
        // ejecutar la función correspondiente a 'id_propio'
        if (id_propio == id_camarero)
            funcion_camarero(); // es el camarero

        else if (id_propio % 2 == 0) // si es par
            funcion_filosofos(id_propio); // es un filósofo

        else // si es impar
            funcion_tenedores(id_propio); // es un tenedor
    }
}
```

```

else{
    if (id_propio == 0){ // solo el primero escribe error, indep. del rol
        cout << "el número de procesos esperados es: " << num_procesos <<
endl
        << "el número de procesos en ejecución es: " << num_procesos_actual <<
endl
        << "(programa abortado)" << endl;
    }
}

MPI_Finalize();
return 0;
}

```

Práctica 4. Implementación de Sistemas de Tiempo Real

1. Nueva funcionalidad en ejecutivo1.cpp para informar retrasos

```
// Datos de las tareas:
// -----
// Ta. T      C
// -----
// A 250 100
// B 250 80
// C 500 50
// D 500 40
// E 1000 20
// -----
//
// Planificación (con Ts == 250 ms)
// *-----*-----*-----*-----*
// | A B C | A B D E | A B C | A B D |
// *-----*-----*-----*-----*
//
// -----

#include <string>
#include <iostream> // cout, cerr
#include <thread>
#include <chrono> // utilidades de tiempo
#include <ratio> // std::ratio_divide

using namespace std;
using namespace std::chrono;
using namespace std::this_thread;

// tipo para duraciones en segundos y milisegundos, en coma flotante:
//typedef duration<float,ratio<1,1>> seconds_f ;
typedef duration<float,ratio<1,1000>> milliseconds_f ;

// -----
// tarea genérica: duerme durante un intervalo de tiempo (de determinada duración)

void Tarea(const std::string & nombre, milliseconds tcomputo){
    cout << " Comienza tarea " << nombre << " (C == " << tcomputo.count() << " ms.) ... ";
    sleep_for(tcomputo);
    cout << "fin." << endl ;
}

// -----
```

Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo espacio



Necesito concentración

ali ali oohh
esto con 1 coin me
lo quito yo...

WUOLAH

// tareas concretas del problema:

```
void TareaA() { Tarea( "A", milliseconds(100) ); }
void TareaB() { Tarea( "B", milliseconds( 80) ); }
void TareaC() { Tarea( "C", milliseconds( 50) ); }
void TareaD() { Tarea( "D", milliseconds( 40) ); }
void TareaE() { Tarea( "E", milliseconds( 20) ); }
```

// -----
// implementación del ejecutivo cíclico:

```
int main(int argc, char *argv[]){
    // Ts = duración del ciclo secundario (en unidades de milisegundos, enteros)
    const milliseconds Ts_ms(250);

    // ini_sec = instante de inicio de la iteración actual del ciclo secundario
    time_point<steady_clock> ini_sec = steady_clock::now();

    while(true){ // ciclo principal
        cout << endl
        << "-----" << endl
        << "Comienza iteración del ciclo principal." << endl;

        for (int i=1; i<=4; i++){ // ciclo secundario (4 iteraciones)
            cout << endl << "Comienza iteración " << i << " del ciclo secundario." << endl;

            switch(i){
                case 1 : TareaA(); TareaB(); TareaC(); break;
                case 2 : TareaA(); TareaB(); TareaD(); TareaE(); break;
                case 3 : TareaA(); TareaB(); TareaC(); break;
                case 4 : TareaA(); TareaB(); TareaD(); break;
            }

            // calcular el siguiente instante de inicio del ciclo secundario
            ini_sec += Ts_ms;

            // esperar hasta el inicio de la siguiente iteración del ciclo secundario
            sleep_until(ini_sec);
            time_point<steady_clock> fin_sec = steady_clock::now();
            steady_clock::duration duracion = fin_sec - ini_sec;

            cout << "Duración esperada: " << Ts_ms.count() << " milisegundos" << endl;
            cout << "Retraso: " << milliseconds_f(duracion).count() << " milisegundos" << endl;
        }
    }
}
```

2. Nuevo ejemplo

```
// -----
// Archivo: ejecutivo2.cpp
// Implementación del segundo ejemplo de ejecutivo cíclico:
//
// Datos de las tareas:
// -----
// Ta. T      C
// -----
// A   500    100
// B   500    150
// C  1000    200
// D  2000    240
// -----
//
// Planificación (con Ts == 500 ms)
// *-----*-----*-----*-----*
// | A B C | A B E | A B C | A B   |
// *-----*-----*-----*-----*
//
// -----

#include <string>
#include <iostream> // cout, cerr
#include <thread>
#include <chrono>   // utilidades de tiempo
#include <ratio>    // std::ratio_divide

using namespace std;
using namespace std::chrono;
using namespace std::this_thread;

// tipo para duraciones en segundos y milisegundos, en coma flotante:
//typedef duration<float,ratio<1,1>>  seconds_f ;
typedef duration<float,ratio<1,1000>> milliseconds_f;

// -----
// tarea genérica: duerme durante un intervalo de tiempo (de determinada duración)

void Tarea(const std::string & nombre, milliseconds tcomputo){
    cout << "  Comienza tarea " << nombre << " (C == " << tcomputo.count() << " ms.) ... ";
    sleep_for(tcomputo);
    cout << "fin." << endl ;
}

// -----
```

```

// tareas concretas del problema:

void TareaA() { Tarea( "A", milliseconds(100) ); }
void TareaB() { Tarea( "B", milliseconds(150) ); }
void TareaC() { Tarea( "C", milliseconds(200) ); }
void TareaD() { Tarea( "D", milliseconds(240) ); }

// -----
// implementación del ejecutivo cíclico:

int main(int argc, char *argv[]){
    // Ts = duración del ciclo secundario (en unidades de milisegundos, enteros)
    const milliseconds Ts_ms(500);

    // ini_sec = instante de inicio de la iteración actual del ciclo secundario
    time_point<steady_clock> ini_sec = steady_clock::now();

    while(true){ // ciclo principal
        cout << endl
        << "-----" << endl
        << "Comienza iteración del ciclo principal." << endl;

        for (int i=1; i<=4; i++){ // ciclo secundario (4 iteraciones)
            cout << endl << "Comienza iteración " << i << " del ciclo secundario." <<
endl;

            switch(i){
                case 1 : TareaA(); TareaB(); TareaC(); break;
                case 2 : TareaA(); TareaB(); TareaD(); break;
                case 3 : TareaA(); TareaB(); TareaC(); break;
                case 4 : TareaA(); TareaB();          break;
            }

            // calcular el siguiente instante de inicio del ciclo secundario
            ini_sec += Ts_ms;

            // esperar hasta el inicio de la siguiente iteración del ciclo secundario
            sleep_until(ini_sec);
            time_point<steady_clock> fin_sec = steady_clock::now();
            steady_clock::duration duracion = fin_sec - ini_sec;

            cout << "Duración esperada: " << Ts_ms.count() << " milisegundos" << endl;
            cout << "Retraso: " << milliseconds_f(duracion).count() << " milisegundos" << endl;
        }
    }
}

```


Importante

Puedo eliminar la publi de este documento con 1 coin

¿Cómo consigo coins? → Plan Turbo: barato
→ Planes pro: más coins

pierdo
espacio



Necesito
concentración

ali ali ooh
esto con 1 coin me
lo quito yo...

WUOLAH

Cuestiones:

- ¿Cuál es el mínimo tiempo de espera que queda al final de las iteraciones del ciclo secundario con tu solución ?

10 milisegundos

- ¿Sería planificable si la tarea D tuviese un tiempo cómputo de 250 ms?

Teóricamente sí pero el primer ciclo secundario tendría una duración exacta de 250 ms, considerando los retrasos que se dan no se cumpliría.